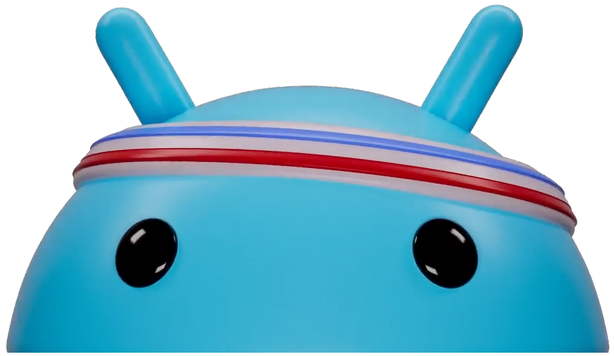


ViewModel and Navigation

ДВФУ · ИМКТ · Департамент МиКМ · Android-разработка · весна-2026

Agenda

- Thinking in Compose(recap)
- ViewModel
- Navigation Compose

Outcome

- State holder -> ViewModel
- If/else экраны -> Navigation Compose
- Всё ещё без сети и БД

Recap: Thinking in Compose

- @Composable функции
- UI = функция от state
- Обновление UI происходит через recomposition
- Удобная формула: data down, events up

UDF: “state вниз, события вверх”

- Вниз: UiState (данные для отрисовки)
- Вверх: Events (коллбеки/методы)
- Плюсы:
  - предсказуемо
  - проще тестировать
  - UI становится “тупым”

State hoisting

- Hoist state к lowest common ancestor
- Из state owner наружу:
  - immutable state
  - events для изменения state

https://developer.android.com/develop/ui/compose/state-hoisting

ViewModel, Navigation - это просто инструменты, чтобы эту формулу сохранить на большом проекте.

ViewModel

ViewModel — это screen-level state holder, то есть объект, который хранит состояние экрана, обрабатывает события пользователя и содержит связанную с экраном логику, отдавая UI уже готовый UiState. Его главное практическое свойство в Android — он переживает configuration changes вроде поворота экрана, поэтому состояние не теряется при пересоздании Activity/Fragment.

https://developer.android.com/topic/libraries/architecture/viewmodel

State holder: “мозги” приложения

- State holder = место, где:
  - хранится UiState
  - обрабатываются события
  - вычисляются видимые списки / фильтры
- Реализация:
  - plain class
  - или ViewModel

remember / rememberSaveable / ViewModel

- remember: живёт пока composable остаётся в композиции
- rememberSaveable: переживает пересоздание, когда это возможно
- ViewModel: основной способ отдавать UI state в Compose

remember / rememberSaveable / ViewModel

- rememberSaveable: удобен для простого UI-state (в пределах экрана)
- ViewModel: screen-level state holder, переживает конфиг-изменения

plain class -> ViewModel

- Было: val holder = remember { Holder() }
- Стало: val holder: Holder = viewModel()
- И class Holder : ViewModel()

Но можно же в rememberSaveable plain class?

- **rememberSaveable** складывает данные в saved instance state (Bundle). Это “общий мешок” для всего Activity, у него есть жёсткие ограничения по размеру, и Google прямо предупреждает: не хранить большие/сложные объекты и списки, иначе можно словить TransactionTooLarge. Вместо этого хранить минимум (id/keys) и восстанавливать остальное другим способом.

https://developer.android.com/develop/ui/compose/state-saving

А зачем это вообще?

- Plain class = “состояние живёт, пока живёт composable”.
- ViewModel = “состояние живёт, пока живёт экран (и переживает поворот)

Где должна жить логика

- UI:
  - рисует
  - вызывает callbacks
- State holder / ViewModel:
  - решает “что делать”
  - меняет state

Выделение viewmodel

- ViewModel выделяют по границе независимого UI-состояния, а не по предметной области и не по числу экранов/composable.

- -> Одна ViewModel - для одного хозяина состояния.

Выделение viewmodel

- если разные части UI всегда меняются вместе и логически составляют одно целое - обычно хватает одной ViewModel
- если какая-то часть должна жить отдельно, загружаться отдельно, обрабатываться отдельно, переживать свои события отдельно — тогда возможна отдельная ViewModel
-

Выделение viewmodel

- У этого куска UI есть своё состояние?
  Например: loading/error/data, выбранный элемент, форма, пагинация.
- Это состояние меняется по своим правилам?
  Не вместе со всем экраном, а отдельно.
- Можно ли эту часть развивать отдельно от остального экрана?
  Например, переиспользовать, вынести, тестировать отдельно.

Navigation

Почему нам нужна навигация

- if (selectedId == null) … else … - ок для демо
- Но дальше нужно:
  - back stack
  - аргументы
  - единое описание графа экранов
  - depplink
- Решение: Navigation Compose

https://developer.android.com/develop/ui/compose/navigation

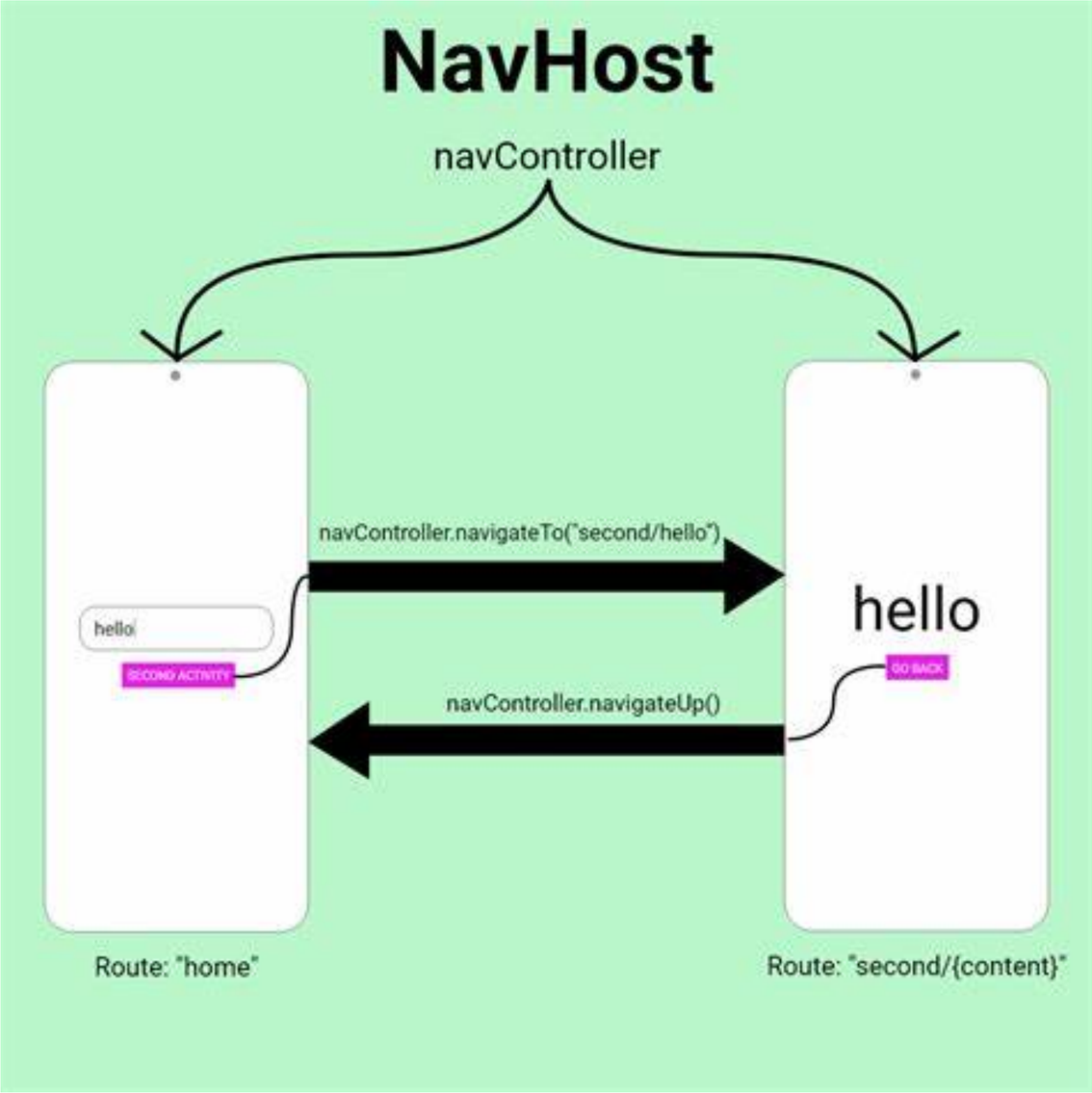
Navigation Compose: словарь

- NavController - управляет переходами
- NavHost - граф навигации
- route - строка маршрута (например, "detail/{id}")

https://developer.android.com/develop/ui/compose/navigation

Аргументы: передаём ID, не объект

- ✅ detail/7
- ❌ “передать весь Anime”
  - Почему:
  - проще
  - безопаснее
  - меньше связанности



Back stack и “назад”

- navigate("detail/7") -> пуш в стек
- “назад” -> popBackStack()
- “вверх” (up) - обычно то же самое в простом кейсе

Modifier

Modifier — это специальный объект, который позволяет «декорировать» (изменить) UI-элемент (composable): его размер, поведение, расположение, оформление.

Modifier

Modifier - конструктор поведения

Modifier - это способ изменить или дополнить поведение и внешний вид composable-элемента.

Через него вы говорите «этот элемент должен быть таким-то»:

- как он выглядит (size, padding, background)
- как он располагается (align, fillMaxWidth)
- как он реагирует на жесты/клики (clickable)
- как он обрабатывает ввод (scroll, draggable)
- какие эффекты получает (shadow, border)

Фактически это цепочка инструкций, применяемая к UI-элементу.

Modifier

```
modifier = Modifier
    .background(Color.Red)
    .padding(16.dp)

// фон на всю область + "внутренний" отступ
```

```
modifier = Modifier
    .padding(16.dp)
    .background(Color.Red)

// фон только вокруг содержимого, "отступ" вокруг карточки прозрачный
```

Полный список Modifier’s

https://developer.android.com/develop/ui/compose/modifiers-list

mutableStateOf vs Flow

Не использовать Flow

(Пока что)

https://developer.android.com/topic/architecture/ui-layer/state-production